a Software stack

ACPU Studio · desktop UI, 3 platforms formula console, device view, run history

▼

SDK · Python / Node / HTTP one-line calls: forecast(), assimilate(), lyapunov()

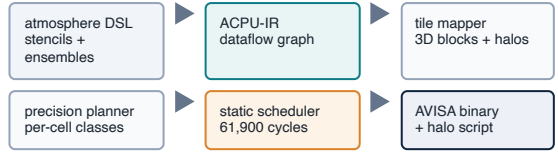
▼

ACPU driver + runtime queue, DMA, telemetry, ECC counters, 9771/tcp

▼

ACPU-1 device 64 AVTs, 128 MB L3, 48 GB HBM3E

b Compilation flow



No dynamic scheduling: the cycle count of every tile is known before the run starts, which is what makes the 71% utilisation reproducible and auditable.

c One-line calls

```
import acpu
dev = acpu.ACPUDevice()

r = dev.forecast(
  model="l96", members=64,
  lead_days=10, precision="adaptive")

print(r.rmse, r.energy_j, r.seconds)
```

d Driver interface

call	meaning
ACPI_DEV_INFO	64 tiles, clocks, temperature
ACPI_LOAD	AVISA binary + constants
ACPI_RUN	start a job, returns job id
ACPI_POLL	progress, energy, Tj
ACPI_READ	DMA result buffers
ACPI_ECC	corrected / uncorrected counts

The virtual device in the app implements the same seven calls, so code written against it runs unchanged on silicon.

The same call exists in JavaScript, on the CLI and over HTTP; the console in the app parses `forecast(members=64, lead_days=10)`.